

EXPERIMENT 1: Feature Descriptor Comparison (SIFT vs Harris)

Aim

To compare Harris Corner Detection and SIFT feature extraction on the same image and analyze their feature detection performance.

Software Required

- Python
- OpenCV
- NumPy
- Matplotlib

Brief Theory

Harris Corner Detection detects points where image intensity changes significantly in two directions.

SIFT detects distinctive keypoints and generates descriptors that are robust to scale and rotation.

Procedure

1. Read the input image and convert it into grayscale.
2. Apply Harris Corner Detection to identify corner points.
3. Apply SIFT to detect keypoints and descriptors.
4. Display both feature detection results.
5. Compare the detected features based on quality, robustness, and suitability for image matching.

Program

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("image.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Harris
gray_float = np.float32(gray)
harris = cv2.cornerHarris(gray_float, 2, 3, 0.04)
```

```
harris = cv2.dilate(harris, None)
```

```
harris_img = img.copy()
```

```
harris_img[harris > 0.01 * harris.max()] = [0, 0, 255]
```

```
# SIFT
```

```
sift = cv2.SIFT_create()
```

```
keypoints, descriptors = sift.detectAndCompute(gray, None)
```

```
sift_img = cv2.drawKeypoints(
```

```
    img, keypoints, None,
```

```
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
```

```
)
```

```
print("SIFT Keypoints:", len(keypoints))
```

```
plt.figure(figsize=(12, 5))
```

```
plt.subplot(1, 3, 1)
```

```
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
```

```
plt.title("Original")
```

```
plt.axis("off")
```

```
plt.subplot(1, 3, 2)
```

```
plt.imshow(cv2.cvtColor(harris_img, cv2.COLOR_BGR2RGB))
```

```
plt.title("Harris Corners")
```

```
plt.axis("off")
```

```
plt.subplot(1, 3, 3)
```

```
plt.imshow(cv2.cvtColor(sift_img, cv2.COLOR_BGR2RGB))
```

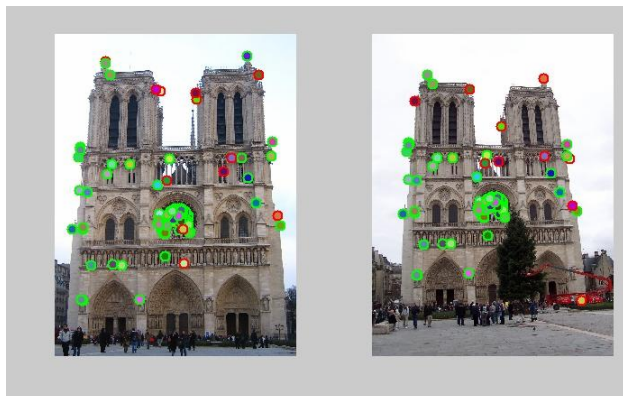
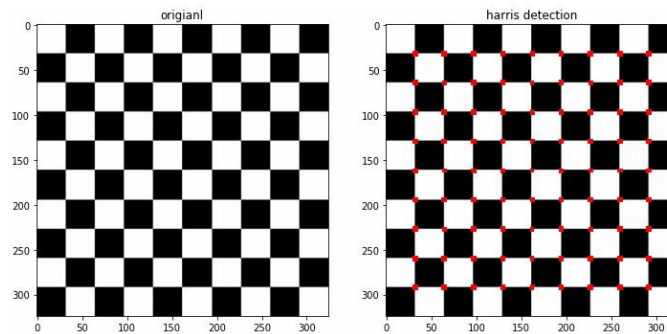
```
plt.title("SIFT Keypoints")
```

```
plt.axis("off")
```

```
plt.tight_layout()
```

```
plt.show()
```

Output :



Observation

Method	Feature Detection	Descriptor	Robustness
Harris	Corners	No	Moderate
SIFT	Keypoints	Yes	High

Result

SIFT provided more robust features and descriptors suitable for image matching, while Harris was effective for detecting corner points.

EXPERIMENT 2: Image Enhancement for Feature Detection

Aim

To study the effect of histogram equalization and contrast adjustment on feature detection.

Brief Theory

Image enhancement improves image quality and makes important features such as edges, corners, and textures more visible. Histogram equalization improves intensity distribution, while contrast adjustment expands the intensity range.

Procedure

1. Read the input image and convert it to grayscale.
2. Apply histogram equalization.
3. Apply contrast adjustment.
4. Perform SIFT feature detection on the original and enhanced images.
5. Compare the number and visibility of detected features.

Program

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("image.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Histogram Equalization
equalized = cv2.equalizeHist(gray)

# Contrast Adjustment
contrast = cv2.normalize(
    gray, None, 0, 255, cv2.NORM_MINMAX
)

sift = cv2.SIFT_create()
```

```
kp1, _ = sift.detectAndCompute(gray, None)
kp2, _ = sift.detectAndCompute(equalized, None)
kp3, _ = sift.detectAndCompute(contrast, None)

result1 = cv2.drawKeypoints(gray, kp1, None)
result2 = cv2.drawKeypoints(equalized, kp2, None)
result3 = cv2.drawKeypoints(contrast, kp3, None)

print("Original:", len(kp1))
print("Equalized:", len(kp2))
print("Contrast:", len(kp3))

plt.figure(figsize=(12, 8))

plt.subplot(2, 3, 1)
plt.imshow(gray, cmap="gray")
plt.title("Original")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(equalized, cmap="gray")
plt.title("Histogram Equalization")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(contrast, cmap="gray")
plt.title("Contrast Adjustment")
plt.axis("off")

plt.subplot(2, 3, 4)
```

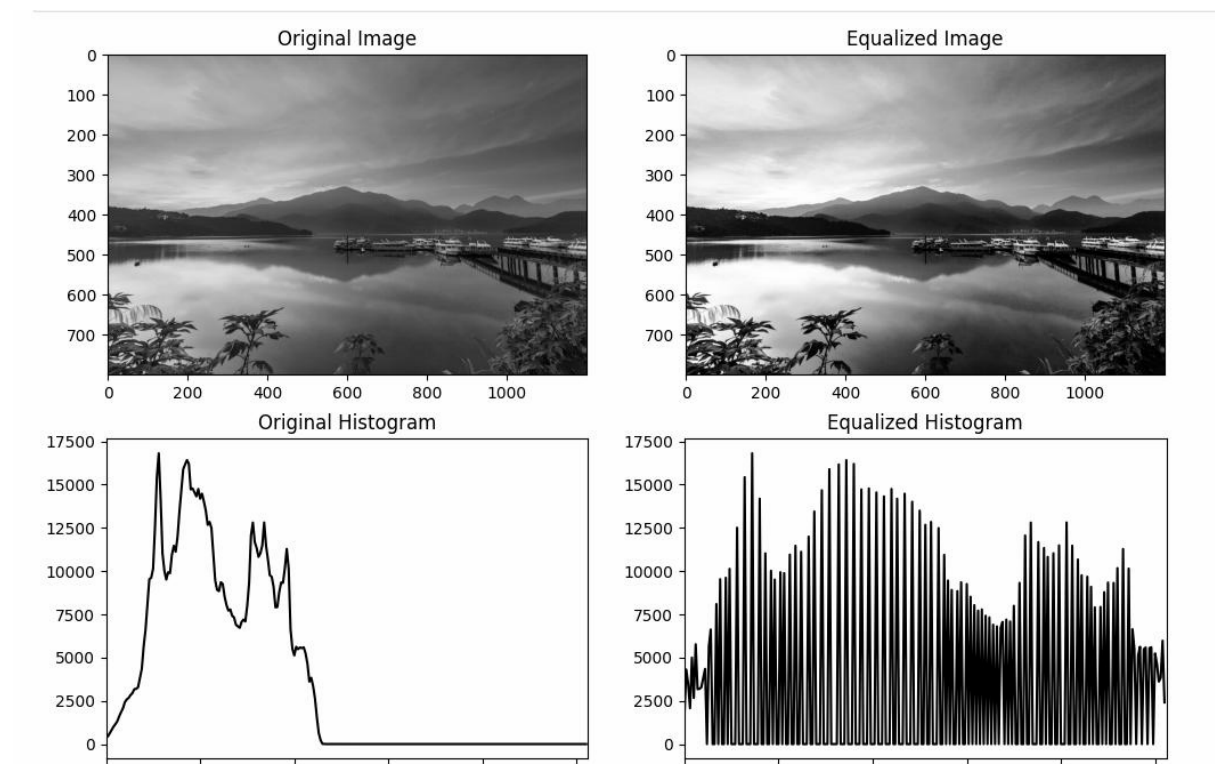
```
plt.imshow(result1, cmap="gray")  
plt.title("Original Features")  
plt.axis("off")
```

```
plt.subplot(2, 3, 5)  
plt.imshow(result2, cmap="gray")  
plt.title("Equalized Features")  
plt.axis("off")
```

```
plt.subplot(2, 3, 6)  
plt.imshow(result3, cmap="gray")  
plt.title("Contrast Features")  
plt.axis("off")
```

```
plt.tight_layout()  
plt.show()
```

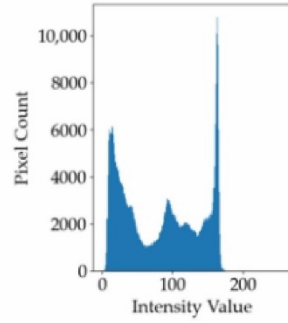
Output



(A) Original Image



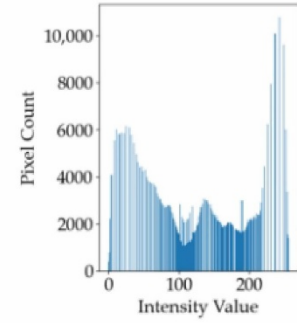
(B) Histogram Before



(C) Image After
Histogram Equalization



(D) Histogram After



Observation

Method	Feature Visibility
Original	Moderate
Histogram Equalization	Improved
Contrast Adjustment	Improved

Result

Image enhancement improved feature visibility and influenced the number and distribution of detected features.

EXPERIMENT 3: Multi-Scale Feature Detection Analysis

Aim

To analyze feature detection at different image scales and study the effect of image resolution on feature stability and computational performance.

Brief Theory

Image scaling changes the resolution or size of an image. Multi-scale feature detection allows features to be studied at different resolutions. SIFT is designed to detect features over different scales.

Procedure

1. Read the input image.
2. Resize the image to 100%, 75%, 50%, and 25%.
3. Apply SIFT feature detection at each scale.
4. Count the detected keypoints.
5. Record processing time and compare the results.

Program

```
import cv2
import time
import matplotlib.pyplot as plt

img = cv2.imread("image.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

scales = [1.0, 0.75, 0.50, 0.25]
sift = cv2.SIFT_create()

plt.figure(figsize=(10, 8))

for i, scale in enumerate(scales):

    width = int(gray.shape[1] * scale)
```



```
height = int(gray.shape[0] * scale)
```

```
resized = cv2.resize(gray, (width, height))
```

```
start = time.time()
```

```
keypoints, descriptors = sift.detectAndCompute(  
    resized, None
```

```
)
```

```
end = time.time()
```

```
result = cv2.drawKeypoints(  
    resized,  
    keypoints,  
    None,  
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS  
)
```

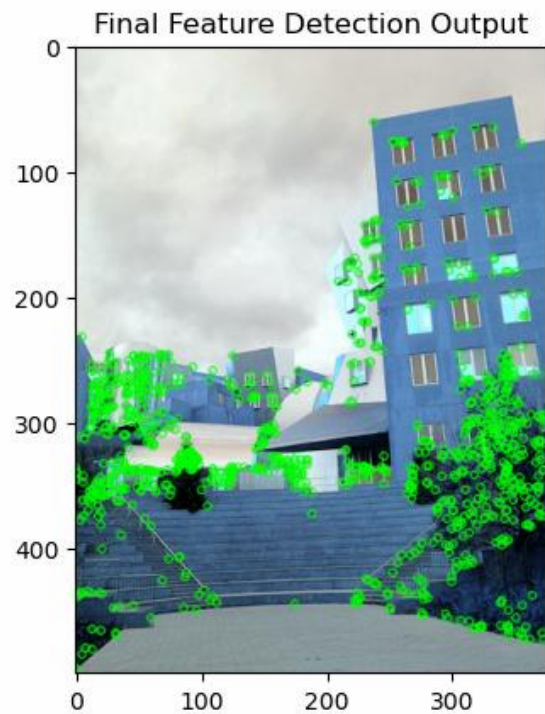
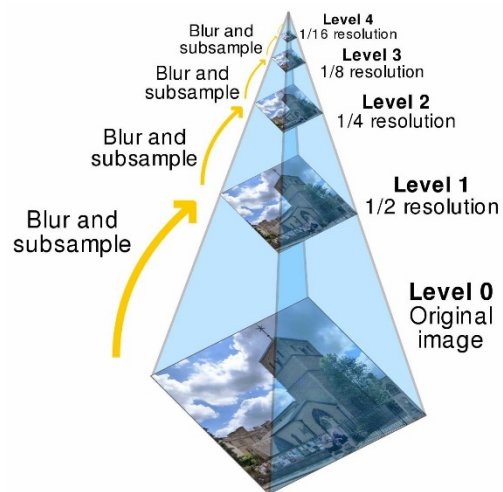
```
print(  
    f"Scale: {scale*100:.0f}% | "  
    f"Keypoints: {len(keypoints)} | "  
    f"Time: {end-start:.5f} sec"  
)
```

```
plt.subplot(2, 2, i + 1)  
plt.imshow(result, cmap="gray")  
plt.title(f"Scale {scale*100:.0f}%")  
plt.axis("off")
```

```
plt.tight_layout()
```

```
plt.show()
```

Output –



Observation

Scale	Keypoints	Time
100%	Your output	Your output
75%	Your output	Your output
50%	Your output	Your output
25%	Your output	Your output

Result

Feature detection was affected by image scale. Higher-resolution images generally preserve more details, while smaller images require less processing time but may lose fine features.

EXPERIMENT 4: Comparative Analysis of Image Filtering Techniques

Aim

To compare Mean, Gaussian, and Median filtering techniques for noise reduction and edge preservation.

Brief Theory

Mean Filter: Replaces each pixel with the average of neighboring pixels.

Gaussian Filter: Uses a Gaussian-weighted average to smooth the image.

Median Filter: Replaces a pixel with the median of its neighborhood and is highly effective for salt-and-pepper noise.

Procedure

1. Read the input image.
2. Convert it into grayscale.
3. Add salt-and-pepper noise.
4. Apply Mean filtering.
5. Apply Gaussian filtering.
6. Apply Median filtering.
7. Compare noise reduction and edge preservation.

Program

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("image.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Add noise
noisy = gray.copy()

amount = 0.04
num_pixels = int(amount * gray.size)
```

```

# Salt noise
coords = [
    np.random.randint(0, i, num_pixels)
    for i in gray.shape
]
noisy[coords[0], coords[1]] = 255

# Pepper noise
coords = [
    np.random.randint(0, i, num_pixels)
    for i in gray.shape
]
noisy[coords[0], coords[1]] = 0

# Filters
mean = cv2.blur(noisy, (5, 5))
gaussian = cv2.GaussianBlur(noisy, (5, 5), 0)
median = cv2.medianBlur(noisy, 5)

plt.figure(figsize=(12, 8))

plt.subplot(2, 3, 1)
plt.imshow(gray, cmap="gray")
plt.title("Original")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(noisy, cmap="gray")
plt.title("Noisy")
plt.axis("off")

```

```
plt.subplot(2, 3, 4)
```

```
plt.imshow(mean, cmap="gray")
```

```
plt.title("Mean Filter")
```

```
plt.axis("off")
```

```
plt.subplot(2, 3, 5)
```

```
plt.imshow(gaussian, cmap="gray")
```

```
plt.title("Gaussian Filter")
```

```
plt.axis("off")
```

```
plt.subplot(2, 3, 6)
```

```
plt.imshow(median, cmap="gray")
```

```
plt.title("Median Filter")
```

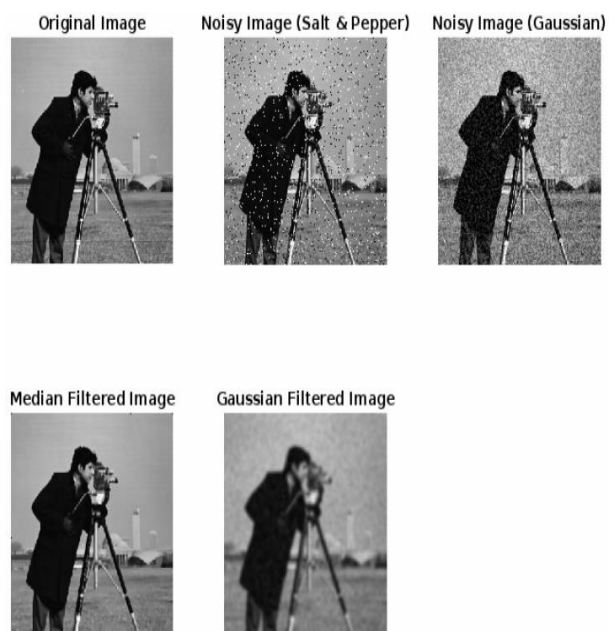
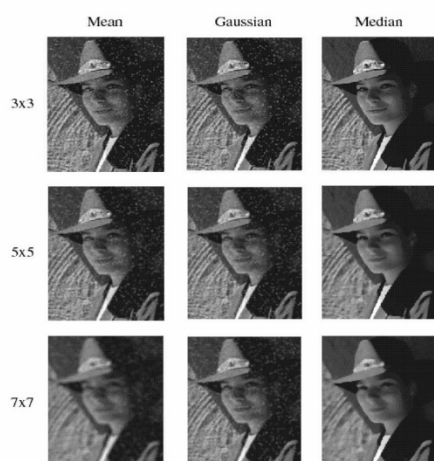
```
plt.axis("off")
```

```
plt.tight_layout()
```

```
plt.show()
```

Output :

Comparison: salt and pepper noise



You can also paste the **single combined screenshot** generated by the program.

Observation

Filter	Noise Reduction	Edge Preservation
Mean	Good	Low
Gaussian	Good	Moderate
Median	Very Good	High

Result

Median filtering was most effective for salt-and-pepper noise, while Gaussian filtering produced smooth results with moderate edge preservation.

EXPERIMENT 5: Complete Object Detection Workflow

Aim

To develop a complete object detection workflow using preprocessing, edge detection, feature extraction, contour detection, and shape detection.

Brief Theory

The object detection pipeline consists of:

Input Image



Grayscale



Preprocessing



Canny Edge Detection



Contour Detection



Feature Extraction



Shape Detection



Final Object Detection

Procedure

1. Read the input image.
2. Convert it to grayscale.
3. Apply Gaussian filtering.
4. Apply Canny edge detection.
5. Detect contours.
6. Extract contour features.
7. Identify the object shape.
8. Draw bounding boxes and labels around detected objects.
9. Display the final result.

Program

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("objects.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Preprocessing
blur = cv2.GaussianBlur(gray, (5, 5), 0)

# Edge Detection
edges = cv2.Canny(blur, 50, 150)

# Contours
contours, _ = cv2.findContours(
    edges,
    cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE
)

output = img.copy()

for contour in contours:

    area = cv2.contourArea(contour)

    if area < 500:
        continue
```



```
perimeter = cv2.arcLength(contour, True)
```

```
approx = cv2.approxPolyDP(  
    contour,  
    0.04 * perimeter,  
    True  
)
```

```
vertices = len(approx)
```

```
if vertices == 3:
```

```
    shape = "Triangle"
```

```
elif vertices == 4:
```

```
    x, y, w, h = cv2.boundingRect(approx)
```

```
    if 0.95 <= w / float(h) <= 1.05:
```

```
        shape = "Square"
```

```
    else:
```

```
        shape = "Rectangle"
```

```
elif vertices == 5:
```

```
    shape = "Pentagon"
```

```
else:
```

```
    shape = "Circle"
```

```
x, y, w, h = cv2.boundingRect(contour)
```

```
cv2.drawContours(
```

```
    output, [contour], -1, (0, 255, 0), 2
)
```

```
cv2.rectangle(
    output,
    (x, y),
    (x+w, y+h),
    (255, 0, 0),
    2
)
```

```
cv2.putText(
    output,
    shape,
    (x, y-10),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.7,
    (0, 0, 255),
    2
)
```

```
plt.figure(figsize=(12, 8))
```

```
plt.subplot(2, 3, 1)
```

```
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
```

```
plt.title("Original")
```

```
plt.axis("off")
```

```
plt.subplot(2, 3, 2)
```

```
plt.imshow(gray, cmap="gray")
```

```
plt.title("Grayscale")
```

```
plt.axis("off")
```

```
plt.subplot(2, 3, 3)
```

```
plt.imshow(blur, cmap="gray")
```

```
plt.title("Preprocessed")
```

```
plt.axis("off")
```

```
plt.subplot(2, 3, 4)
```

```
plt.imshow(edges, cmap="gray")
```

```
plt.title("Canny Edges")
```

```
plt.axis("off")
```

```
plt.subplot(2, 3, 5)
```

```
plt.imshow(cv2.cvtColor(output, cv2.COLOR_BGR2RGB))
```

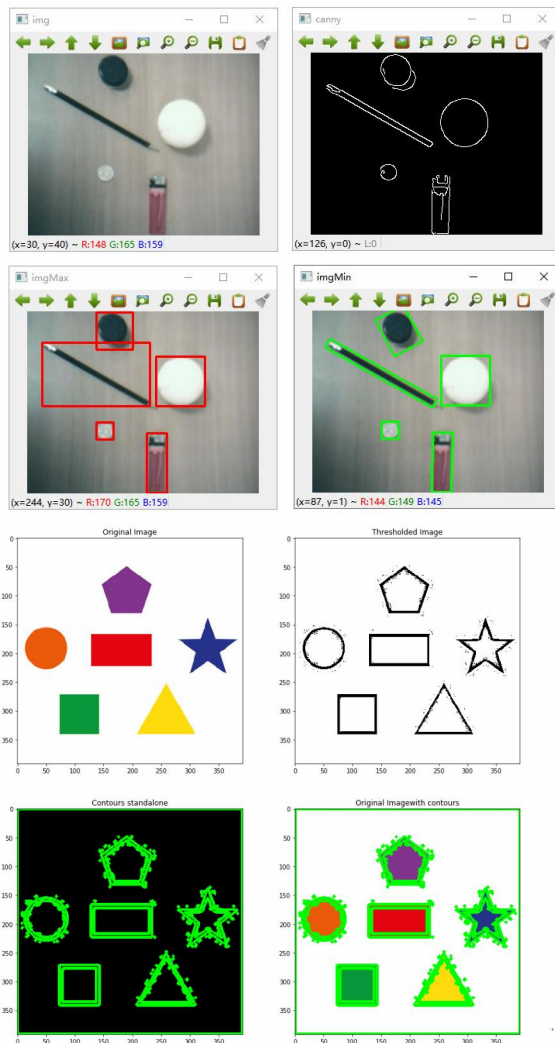
```
plt.title("Final Object Detection")
```

```
plt.axis("off")
```

```
plt.tight_layout()
```

```
plt.show()
```

Output :



Observation

Stage	Observation
Original	Objects are visible
Grayscale	Intensity information obtained
Preprocessing	Noise reduced
Canny	Object edges detected
Contours	Object boundaries identified
Shape Detection	Objects classified

Result

The complete image-processing pipeline successfully detected objects using preprocessing, Canny edge detection, contour extraction, and shape analysis.

